

Evaluation Cascade: Production-Style LLM Evaluation and Regression Platform

Requirements & Specifications Document

Version 1.0 — Proposed MVP

Status: Draft for implementation

Primary audience: Evaluation engineering, ML infrastructure, and software engineering hiring teams

1. Purpose and Positioning

Evaluation Cascade is a reproducible evaluation and regression platform for applications that use small, locally hosted language models.

The project will demonstrate the ability to integrate model evaluation into a software-development workflow:

- Define versioned evaluation cases
- Run an application against a controlled test suite
- Apply deterministic and model-assisted evaluators
- Store results with complete provenance
- Compare a candidate implementation against a baseline
- Detect quality, safety, latency, and reliability regressions
- Produce an actionable release decision
- Route uncertain or high-risk cases for review

The project will initially support a linguistically grounded evaluation workload derived from the existing Cognitive-Eval work. Its core interfaces will be application-agnostic so that it can pivot to, or additionally support, a structured extraction pipeline without requiring a redesign of the evaluation infrastructure.

The central engineering question is:

Can this system provide reliable, explainable, and reproducible evidence that a change to an LLM application is safe to release?

The linguistic benchmark is a demonstration workload, not the primary product identity.

2. Project Goals

2.1 Primary goals

The MVP shall:

1. Provide a reusable evaluation framework for LLM applications.
2. Support deterministic verification as the first evaluation layer.
3. Preserve the existing rule graph and verifiers as one evaluator implementation.
4. Support versioned datasets, application configurations, prompts, models, and evaluators.
5. Execute baseline and candidate evaluations reproducibly.
6. Detect aggregate and slice-level regressions.
7. Apply configurable release gates.
8. Store per-example evidence explaining pass, fail, or review outcomes.
9. Integrate with continuous integration.
10. Present results through a small dashboard and machine-readable reports.
11. Support a structured extraction application as a second workload or pivot path.

2.2 Secondary goals

The MVP should:

- Support Ollama-served open-weight models.
- Support deterministic evaluators for structured outputs.
- Support an optional model-assisted evaluator for qualities that cannot be checked deterministically.
- Support human-review queues for uncertain or high-severity cases.
- Measure operational metrics such as latency, errors, retries, and token usage where available.
- Demonstrate at least one intentionally introduced regression being detected and blocking release.

2.3 Non-goals

The MVP will not attempt to:

- Build a general-purpose autonomous agent framework.
- Prove a broad theory of LLM reasoning or self-correction.
- Perform fine-tuning, weight updates, activation steering, or mechanistic interpretability.
- Produce a large academic benchmark.
- Support hosted frontier models as a primary requirement.
- Build a general-purpose annotation platform.
- Guarantee semantic correctness through an LLM judge alone.

- Optimize the models themselves.
- Expand into many linguistic phenomena before the evaluation workflow is complete.
- Build an elaborate production dashboard.
- Implement OWL export or other secondary knowledge-representation formats.
- Claim that repair or self-correction improves LLM reasoning in general.

3. Primary Users and Use Cases

3.1 Primary user

The primary user is an engineer proposing a change to an LLM application, including a change to:

- Model
- Prompt
- Decoding configuration
- Retrieval configuration
- Output schema
- Parsing logic
- Evaluation logic
- Application code

3.2 Core use case: pull-request evaluation

An engineer submits a change. CI runs a fast evaluation suite and compares the candidate against a stored baseline.

The system reports:

- Whether the candidate passed release gates
- Which metrics changed
- Which slices regressed
- Which individual cases failed
- Whether failures are critical
- Whether the candidate requires human review

3.3 Core use case: full evaluation run

An engineer runs a complete evaluation locally or in CI and receives:

- A run identifier

- Reproducibility metadata
- Per-example outputs
- Per-evaluator results
- Aggregate metrics
- Slice-level metrics
- Baseline comparison
- Release recommendation

3.4 Core use case: structured extraction

The application receives unstructured text and produces a structured record.

Example:

```
{  
  "category": "account_access",  
  "priority": "high",  
  "requires_human_review": true,  
  "summary": "The user cannot log in after changing their password."  
}
```

The platform evaluates:

- Schema validity
- Required-field recall
- Label accuracy
- Numeric and date accuracy
- Unsupported or hallucinated fields
- Internal consistency
- Latency and failure rate
- Human-review routing

3.5 Secondary use case: linguistically grounded evaluation

The existing Cognitive-Eval workload will serve as an initial adapter. It will evaluate model outputs using:

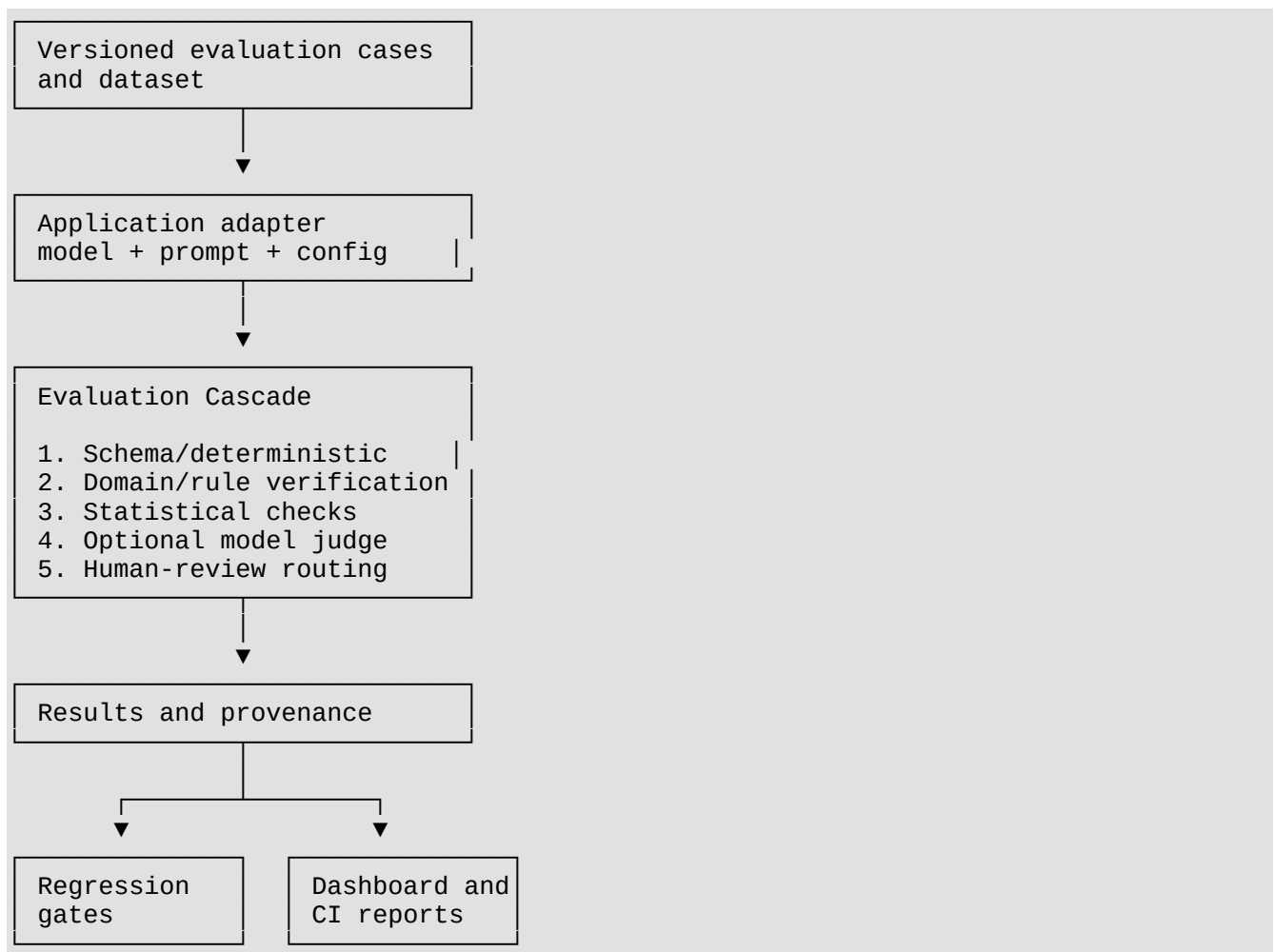
- Existing rule-graph nodes
- Existing deterministic verifiers
- Existing audit information
- Existing linguistic error categories

This workload demonstrates the platform’s ability to support domain-specific, evidence-backed evaluators.

4. System Scope

The system will consist of six major components:

1. Application adapter
2. Evaluation-case and dataset manager
3. Evaluation cascade
4. Results and provenance store
5. Regression and release-gate engine
6. Reports and dashboard



5. Functional Requirements

FR-1: Application adapter interface

The platform shall support applications through a common interface.

Each adapter shall:

- Accept a versioned evaluation case.
- Invoke the target application.
- Return a structured output.
- Record runtime metadata.
- Report application errors, timeouts, and retries.

A minimal Python interface may be:

```
class ApplicationAdapter(Protocol):
    def run(self, case: dict, config: dict) -> dict:
        ...
```

The returned record shall distinguish:

- Successful application output
- Timeout
- Runtime error
- Invalid response
- Retry exhaustion

FR-2: Versioned evaluation cases

Each evaluation case shall have:

- Stable identifier
- Input
- Expected output information
- Tags or slices
- Severity
- Evaluation-specific metadata
- Dataset version

Example:

```
{
  "id": "ticket-0042",
  "input": "I changed my password and now I cannot log in.",
  "required_fields": {
    "category": "account_access",
    "requires_human_review": true
  },
  "tags": ["authentication", "high-risk"],
  "severity": "high"
}
```

The system shall support:

- Dataset versioning
- Addition and removal of cases
- Case-level provenance
- Regression tracking when cases change
- Separate smoke and full datasets

FR-3: Evaluation cascade

The cascade shall execute evaluators in a defined order. Each evaluator shall return a standardized result.

```
class EvaluationResult(TypedDict):
    status: Literal["pass", "fail", "review", "error", "not_applicable"]
    score: float | None
    category: str | None
    severity: str | None
    evidence: dict
    evaluator_name: str
    evaluator_version: str
```

The cascade shall record:

- Which evaluators ran
- Which evaluator produced the final status
- The reason for escalation
- Evidence used for the decision
- Evaluation latency
- Any evaluator errors

FR-4: Deterministic verification

The platform shall support deterministic evaluators for:

- JSON and schema validity

- Required fields
- Allowed labels
- Data types
- Date and number formats
- Exact values
- Internal consistency
- Required facts
- Forbidden claims
- Citation validity
- Domain-specific rules

The existing Cognitive-Eval rule graph and verifiers shall be adapted as a domain-specific evaluator rather than hard-coded into the platform core.

FR-5: Rule-graph evaluator

The rule-graph evaluator shall:

- Accept an application output and evaluation case.
- Identify the applicable rule node.
- Run the associated verifier.
- Return pass/fail status.
- Return the rule identifier.
- Return a human-readable explanation.
- Return supporting citation or provenance metadata.
- Record the verifier version.

The rule graph shall remain reusable and independently testable.

FR-6: Statistical and anomaly checks

The platform may support statistical checks for:

- Output-length changes
- Label-distribution changes
- Novel output clusters
- Increased error concentration

- Latency distribution changes
- Unexpected review rates
- Model-output drift across versions

Statistical checks shall not silently override deterministic failures. They shall produce warnings, review statuses, or regression signals.

FR-7: Optional model-assisted evaluation

The platform may support model-assisted evaluation for properties not fully captured by deterministic checks, such as:

- Relevance
- Summary completeness
- Factual consistency
- Explanation quality
- Whether a response follows a rubric

Every model-assisted result shall record:

- Judge model
- Judge prompt
- Judge configuration
- Judge version
- Evaluation rubric
- Raw judge output
- Parsed score
- Confidence or uncertainty, if available

Model-assisted evaluation shall not be the sole release gate for critical fields unless supported by an independent validation mechanism.

FR-8: Human-review routing

The platform shall identify cases for review when:

- Deterministic and model-assisted evaluators disagree.
- A high-severity case fails.
- A result is near a release threshold.
- An evaluator encounters an unsupported case.

- A new failure category appears.
- A statistical check detects an unusual change.

The dashboard shall display:

- Input
- Application output
- Baseline output
- Evaluation results
- Failure evidence
- Relevant rule or rubric
- Recommended review reason

FR-9: Run reproducibility

Every evaluation run shall record:

- Run identifier
- Timestamp
- Git commit
- Application version
- Model identifier
- Model configuration
- Prompt version
- Dataset version
- Evaluator versions
- Runtime configuration
- Hardware information where practical
- Random seed where applicable
- Temperature and sampling parameters
- Dependency lockfile or environment identifier

FR-10: Baseline comparison

The system shall support comparison between:

- Current candidate run

- Selected baseline run
- Previous release
- Multiple model configurations where applicable

It shall calculate:

- Absolute metric difference
- Relative metric difference
- Number of newly failing cases
- Number of repaired cases
- Number of changed outcomes
- Slice-level differences
- Operational-metric differences

FR-11: Regression detection

The system shall detect:

- Metric drops below minimum thresholds
- Candidate regressions beyond configured tolerances
- New critical failures
- Increased application error rates
- Increased timeout rates
- Latency regressions
- Increased human-review rates
- Output-schema regressions
- Slice-specific regressions hidden by aggregate averages

FR-12: Release gates

Release gates shall be configurable.

Example:

```
gates:
  schema_validity:
    minimum: 0.99

  critical_field_accuracy:
    minimum: 0.98

  high_severity_failure_rate:
```

```
    maximum: 0.01

  application_error_rate:
    maximum: 0.02

  p95_latency_seconds:
    maximum: 5.0

  regression_tolerances:
    overall_quality:
      maximum_drop: 0.03

  high_risk_slice:
    maximum_drop: 0.01
```

The gate engine shall return:

- Pass
- Fail
- Warning
- Review required

A release shall fail automatically when a critical gate fails.

FR-13: CI integration

The project shall provide:

- A fast smoke evaluation for pull requests
- A full evaluation command
- Machine-readable JSON output
- Human-readable terminal output
- Nonzero exit status on release-gate failure
- Persisted candidate results
- A summary suitable for CI comments

Example:

```
python -m evalcascade.run \
  --dataset smoke-v1 \
  --baseline release-2026-08-15 \
  --config configs/ci.yaml \
  --fail-on-gate
```

FR-14: Reporting and dashboard

The dashboard shall display:

- Evaluation runs
- Baseline and candidate metrics
- Gate status
- Slice-level metrics
- Newly failed cases
- Repaired cases
- Error categories
- Latency and error distributions
- Evaluation provenance
- Human-review queue

The dashboard shall remain intentionally small and focused on investigation rather than presentation.

6. Evaluation Metrics

6.1 Quality metrics

Depending on the application adapter, the platform shall support:

- Exact accuracy
- Precision, recall, and F1
- Required-field recall
- Schema-validity rate
- Required-fact recall
- Unsupported-claim rate
- Citation-validity rate
- Groundedness
- Task-completion rate
- Refusal or escalation accuracy

6.2 Reliability metrics

The platform shall measure:

- Application error rate
- Evaluator error rate
- Timeout rate

- Retry rate
- Invalid-output rate
- Review-routing rate

6.3 Operational metrics

Where available, the platform shall measure:

- Mean latency
- p50 latency
- p95 latency
- Token usage
- Memory usage
- Throughput
- Estimated cost or local compute burden

6.4 Slice metrics

All key metrics should be available by:

- Dataset tag
- Severity
- Input length
- Task category
- Model
- Model size
- Application version
- Evaluator version
- Linguistic phenomenon for the Cognitive-Eval adapter
- Structured field for the extraction adapter

7. Structured Extraction Pivot

The platform shall be designed so that structured extraction can become the primary demonstration workload.

7.1 Extraction application

The extraction adapter shall:

1. Receive unstructured text.
2. Invoke a local language model.
3. Parse the model output.
4. Validate the output against a schema.
5. Return the structured record and metadata.

7.2 Extraction evaluators

The initial extraction evaluator set shall include:

- JSON parse validation
- Schema validation
- Required-field recall
- Allowed-value validation
- Exact matching for dates, numbers, and identifiers
- Field-level accuracy
- Hallucinated-field detection
- Cross-field consistency
- Human-review routing accuracy

7.3 Extraction release example

A candidate change shall be rejected if:

Overall schema validity improves from 96% to 98%,
but high-severity field accuracy falls from 99% to 96%.

This demonstrates why production evaluation requires more than one aggregate score.

7.4 Adapter independence

The extraction workload shall use the same:

- Application adapter interface
- Dataset format
- Evaluation-result schema
- Run metadata
- Regression engine
- Release-gate engine

- CI integration
- Dashboard

Only the application-specific evaluators should need to change.

8. Cognitive-Eval Adapter

The existing Cognitive-Eval work shall be preserved as the first domain adapter.

It shall contribute:

- Rule graph
- Rule identifiers
- Deterministic verifiers
- `audit_rule()` evidence
- Linguistic phenomenon tags
- Existing model adapters
- Existing test items
- Existing evaluation logs where reproducible

The adapter shall expose results using the common evaluation schema.

Example:

```
{
  "status": "fail",
  "score": 0.0,
  "category": "RULE_FI_C4_MASS",
  "severity": "medium",
  "evidence": {
    "rule_node_id": "RULE_FI_C4_MASS",
    "explanation": "The object remains partitive under the applicable conditions.",
    "citation": "Kiparsky 1998:280"
  },
  "evaluator_name": "cognitive_rule_verifier",
  "evaluator_version": "1.0.0"
}
```

The linguistic workload shall demonstrate:

- Domain-specific verification
- Evidence-backed failure explanations
- Slice-level analysis
- Model comparison
- Regression detection

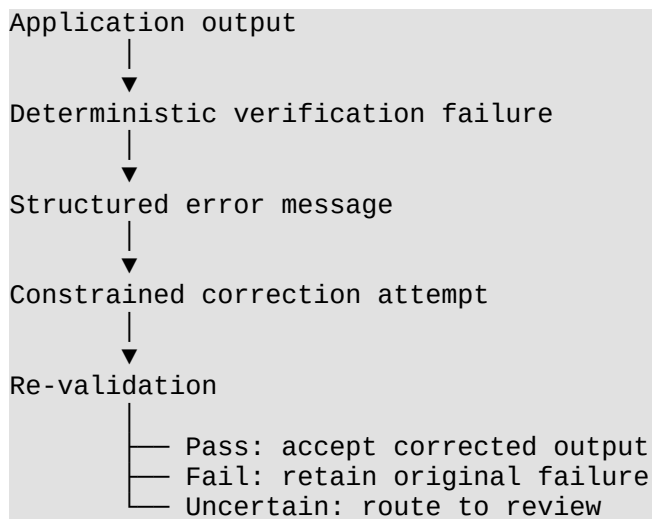
It shall not determine the platform's overall architecture.

9. Verifier-Guided Correction

Correction shall be an optional extension, not an MVP dependency.

If implemented, it shall be framed as **constrained remediation**, not general-purpose cognitive repair.

The correction workflow shall be:



Correction shall be evaluated using:

- Initial validity rate
- Successful correction rate
- False-correction rate
- Critical-field corruption rate
- Retry count
- Added latency
- Added token usage
- Escalation rate

Correction shall not be described as successful merely because the output changes. It is successful only when the corrected output passes independent verification without introducing a more serious error.

10. Technical Requirements

10.1 Required stack

- Python 3.11+
- Pydantic

- pytest
- Inspect AI or an equivalent evaluation runner
- Ollama and/or Hugging Face model adapters
- SQLite for the MVP results store
- Streamlit or a similarly lightweight dashboard
- GitHub Actions
- Docker for reproducible execution where practical

10.2 Architecture requirements

The codebase shall separate:

- Application execution
- Dataset loading
- Evaluation logic
- Result persistence
- Metric aggregation
- Regression analysis
- Release gates
- Presentation

Evaluators shall be independently unit-testable.

10.3 Error handling

The platform shall handle:

- Model timeouts
- Model-server failures
- Malformed model output
- Invalid evaluator results
- Missing baseline runs
- Dataset-version mismatches
- Partial evaluation runs
- Duplicate run identifiers
- Interrupted execution

Failures shall be recorded rather than silently discarded.

11. Data Model

The MVP shall persist at least the following entities:

Evaluation run

- run_id
- created_at
- git_commit
- application_version
- model_config
- dataset_version
- evaluator_versions
- baseline_run_id
- overall_status

Case result

- run_id
- case_id
- raw_application_output
- parsed_output
- application_status
- latency
- evaluator_results
- final_status
- failure_categories
- severity
- review_required

Aggregate metric

- run_id
- metric_name
- value

- slice_name
- slice_value
- threshold
- gate_status

12. Testing Requirements

The platform shall include:

Unit tests

- Dataset parsing
- Schema validation
- Evaluator behavior
- Rule-graph lookups
- Metric calculations
- Regression calculations
- Gate logic
- Serialization and deserialization

Integration tests

- End-to-end evaluation with a mock model
- End-to-end evaluation with Ollama
- Baseline comparison
- CI exit-code behavior
- Dashboard loading from stored results

Regression fixtures

The repository shall include known examples where:

- A valid output passes
- A malformed output fails
- A critical field is wrong
- A rule violation is correctly identified
- A candidate version regresses

- A candidate version improves
- Aggregate metrics improve while a critical slice regresses

13. Milestones

Phase 1: Platform extraction

Duration: 3–5 days

Deliverables:

- Common result schema
- Application adapter interface
- Evaluator interface
- Basic dataset format
- Basic command-line runner
- Existing Cognitive-Eval logic isolated behind an adapter

Phase 2: Deterministic cascade

Duration: 1 week

Deliverables:

- Schema evaluator
- Rule-graph evaluator
- Evaluator dispatcher
- Structured evidence
- Unit tests for all existing rule nodes
- Per-case result persistence

Phase 3: Baseline and regression engine

Duration: 4–6 days

Deliverables:

- Run metadata
- Baseline selection
- Candidate comparison
- Aggregate and slice metrics

- Regression tolerances
- Release gates
- Machine-readable reports

Phase 4: CI integration

Duration: 3–5 days

Deliverables:

- Smoke dataset
- CI command
- Nonzero exit status on gate failure
- Example failed CI run
- Stored evaluation artifacts

Phase 5: Structured extraction adapter

Duration: 1 week

Deliverables:

- Small extraction application
- Versioned extraction dataset
- Pydantic output schema
- Field-level metrics
- Malformed-output tests
- Candidate-versus-baseline demonstration

Phase 6: Dashboard and portfolio documentation

Duration: 4–7 days

Deliverables:

- Run history
- Baseline comparison
- Slice analysis
- Failed-case inspection
- Gate status
- Architecture diagram

- Methodology document
- Error-analysis report
- Reproducible setup instructions

Optional Phase 7: Constrained remediation

Duration: After MVP

Deliverables:

- Deterministic error-to-correction prompt
- Re-validation loop
- Correction metrics
- Cost and latency analysis
- False-correction analysis

14. MVP Success Criteria

The MVP shall be considered portfolio-ready when:

1. A complete evaluation can be run from the command line.
2. The system records application, model, dataset, prompt, evaluator, and code versions.
3. The Cognitive-Eval rule graph operates through the common evaluator interface.
4. At least one structured-output evaluator operates independently of the linguistic adapter.
5. Baseline and candidate runs can be compared.
6. Aggregate and slice-level regressions are detected.
7. Configurable release gates produce pass/fail decisions.
8. CI fails when a critical regression is introduced.
9. At least one intentionally bad change is detected.
10. The system records per-case evidence for failures.
11. The dashboard displays run comparison and failure details.
12. A structured extraction workload can be added without changing the platform core.
13. The repository includes documentation explaining design decisions and limitations.
14. The project reports negative or mixed results honestly rather than presenting only improvements.

15. Portfolio Demonstration

The final demonstration should show three runs:

Run A: Baseline

Schema validity:	96.4%
Critical-field accuracy:	98.1%
p95 latency:	2.4 s
Release status:	PASS

Run B: Improvement

Schema validity:	98.2%
Critical-field accuracy:	98.7%
p95 latency:	2.6 s
Release status:	PASS

Run C: Deliberate regression

Schema validity:	98.8%
Critical-field accuracy:	95.9%
p95 latency:	3.1 s
Release status:	FAIL

Reason:

- Critical-field accuracy fell below the minimum threshold.
- Three high-severity cases newly failed.

The important result is not merely that the model scores well. It is that the system produces a defensible release decision and shows an engineer why.

16. Deferred Work

The following items should be explicitly deferred:

- Large-scale dataset construction
- Broad expansion of linguistic phenomena
- Frontier-model comparisons
- Fine-tuning
- Mechanistic interpretability
- General-purpose agentic loops
- Open-ended self-correction claims
- OWL or other secondary export formats
- Extensive clustering research
- Large human-annotation workflows

- Complex distributed execution
- Production-grade multi-user authentication
- Advanced observability infrastructure
- Autonomous correction without independent re-verification

17. Final Project Description

Evaluation Cascade is a reusable evaluation and regression platform for local LLM applications. It combines deterministic schema and rule-based verification, optional statistical and model-assisted evaluation, evidence-backed failure reporting, versioned experiment tracking, candidate-versus-baseline comparison, slice-level regression analysis, and CI release gates. The platform is demonstrated first with a linguistically grounded verifier and then adapted to a structured extraction workload, showing that the evaluation infrastructure is reusable across applications.

This version preserves the most valuable work from Cognitive-Eval, reduces the research-specific scope, makes the verification cascade central, and leaves constrained correction as an optional extension rather than the project's defining purpose.